

## AP 2026 - Assignment 1

Lukas Vammen - TQR452  
Jimmy Huynh - PLG852  
Karan Inder Singh - GTC626

September 13, 2026

Contents

**Introduction:** 3

**Questions and Answers:** 4

    Question 1 . . . . . 4

    Question 2 . . . . . 4

    Question 3 . . . . . 4

    Question 4 . . . . . 4

    Question 5 . . . . . 4

    Question 6 . . . . . 5

    Question 7 . . . . . 5

    Question 8 . . . . . 5

## Introduction:

The hand-in solution consist of all four tasks.

Thus implementations for 'ForLoop','Lambda','Apply','TryCatch,'and'printExp' have been completed. The code builds without warnings.

For AST\_Tests & Eval\_Tests AI was used to speed-up the creation of tests and add structure to the tests.

We have assumed that the given inputs manually are wrapped by parentheses, and thus it was arbitrary to wrap the string in parentheses.

We think the evaluator is correct, and have reasonable grounds for saying so: the tests are not only "does '+' add", they pin down the decisions that were actually free. In particular 'Apply (static scoping)' would fail if I had used the call site's environment instead of the closure's, 'ForLoop (bindings do not escape)' and 'ForLoop (outer scope visible)' between them catch getting the loop's environment handling wrong, and the Y-combinator factorial test exercises closures, 'If', 'Eql' and recursion end to end. We have also checked all five worked examples from the assignment text and they reproduce exactly. We are less pleased with 'printExp': it is correct but quadratic.

This also leads to an interesting observation about how Haskell represents strings. In Haskell, a string is defined as a linked list of characters. Therefore, a simple concatenation such as  $xs ++ ys$  has time complexity  $O(xs)$ , since Haskell must traverse the entire left-hand list  $xs$  before appending  $ys$ .

We looked into alternative approaches and found a discussion of the ShowS on Stack Overflow [link](#). We did not have time to investigate this approach in detail for this assignment, but it appears to provide a more efficient way of constructing strings by representing intermediate results as functions and composing them, thereby avoiding repeated traversal of already constructed strings.

## Questions and Answers:

### 1. What is the observable evaluation order for Apply, what difference does it make, and which of your test(s) demonstrate this?

The argument is evaluated even though it might not be needed. The order of the alternatives decides which error is reported when more than one error occurs. The evaluation is from left to right, so an error in  $e_1$  beats a failure in  $e_2$ . And a failure in either of  $e_1$  or  $e_2$  will beat the 'Apply: Left Expression not of Type ValFun'.

Two tests demonstrate it. 'Apply (function error wins)' evaluates 'Apply (Var "nope") (Div (CstInt 1) (CstInt 0))', where both halves fail, and gets "Unknown variable: nope" — swapping the first two alternatives would give "Division by zero" instead. 'Apply (argument error beats non-function)' shows the less obvious consequence of the ordering: because the argument's failure is checked *before* the function's shape, 'Apply (CstInt 1) (Div (CstInt 1) (CstInt 0))' reports "Division by zero" rather than "Cannot apply non-function". Laziness also means that when 'e1' fails we never force 'e2' at all, so 'e1's error is returned even if 'e2' would diverge.

### 2. Would it be a correct implementation of eval for TryCatch to first call eval on both subexpressions? Why or why not? If not, under which assumptions might it be a correct implementation?

If we assume that  $e_2$  is harmless, in the sense that there are no observable consequences outside its scope, then it is only a question of efficiency, as there is no reason to evaluate  $e_2$  if  $e_1$  evaluates successfully.

However, it cannot always be the case that  $e_2$  is harmless. What if  $e_2$  throws an exception, diverges, or performs some global state change that should never have occurred? But in the end, it is semantically incorrect for what a try/catch tries to accomplish, so  $e_2$  should never be evaluated together with  $e_1$ .

Therefore, this would not be a correct implementation of try/catch.

### 3. Is it possible for your implementation of eval to go into an infinite loop for some inputs?

The 'ForLoop' alone cannot by itself go into an infinite loop for an input, as we determine a constant integer, and the evaluation makes sure of that. But 'Lambda'/'Apply' can resort to general recursion. Running 'Apply fact (CstInt (-1))' the condition where  $n = 0$  will never occur, as  $n$  will keep decrementing, thus resulting in an infinite loop.

### 4. For each of the following equalities, explain under which circumstances they hold.

a) `Apply (Lambda "v" x) y == x`

b) `TryCatch (TryCatch x y) z == TryCatch x y`

(a) holds when  $x$  does not depend on the lambda parameter  $v$ .

(b) holds when the inner expression `TryCatch x y` cannot throw an exception.

### 5. Suppose we wanted APL to be dynamically scoped. What would need to change in your implementation?

The implementation is currently lexically scoped. The main place where we can make the difference between lexical and dynamic scoping observable is in Apply.

At the moment, a function stores the environment in which it was defined. To make the implementation dynamically scoped, we would remove that stored environment from the function value and instead evaluate the function body using the current environment at the point where the function is applied. This means variable lookup uses the most recent active binding.

For example:

let  $x = 10$  in

let  $f = \lambda y \rightarrow x + y$  in

let  $x = 20$  in  
 $f\ 1$

With lexical scoping, the result is 11, because  $f$  uses the binding  $x = 10$  from the environment where it was defined.

With dynamic scoping, the result is 21, because  $f$  is evaluated in the current environment, where the most recent binding is  $x = 20$ .

## 6. What is the asymptotic complexity of `printExp` on a left-nested expression such as `Add (Add (Add ... ) ...) ...?`

As mentioned in the introduction, the original implementation exhibits quadratic time complexity due to repeated string concatenation. For the specific case of a left-nested expression with  $n$  "Add" nodes, the accumulated cost grows roughly as  $1 + 2 + \dots + n$ , which is  $O(n^2)$ . By using "ShowS", these intermediate concatenations are replaced by constant-time function composition, reducing the complexity for this example to  $O(n)$ .

## 7. State properties that it would be natural for `printExp` to satisfy (you are encouraged to hypothesize additional machinery in order to state such properties).

If we insert enough parentheses to preserve the tree structure, then:  $\text{parseExp} (\text{printExp } e) = e$

A weaker version is that, even if the same syntax tree is not produced, the two expressions should still give the same result:  $\text{eval} (\text{parseExp} (\text{printExp } e)) = \text{eval } e$

If  $e$  is finite, then the printed string should also be finite.

The printed string should follow the precedence and associativity rules of the expression.

If two expressions are the same, then their printed strings should also be the same:

$$e_1 = e_2 \implies \text{printExp } e_1 = \text{printExp } e_2$$

## 8. Give a reason for why `Val` is defined in `APL.Eval` rather than in `APL.AST`.

AST and Eval has two different purposes.

Eval defines the syntax of what the program. It defines what the program is and what the expressions can be used.

AST is part of the semantics, defining what the expressions produces.

As 'ValFun' carries an 'Env', the data structure belongs to the interpreter. Putting 'Val' in 'APL.AST' would drag that runtime representation into the syntax module, and the dependency would run the wrong way — 'APL.Eval' imports 'APL.AST', never the reverse. As it stands, every other consumer of the AST can ignore evaluation entirely: 'printExp' in this very assignment needs 'Exp' and nothing else.